

# Security of Agentic AI and LLM-based Applications: A Systematic Literature Review of Threats, Attack Surfaces, Benchmarks, and Mitigation Strategies

## Abstract

Large language models (LLMs) have evolved from passive text generators into the reasoning core of *agentic* systems that plan, remember, retrieve external knowledge, call tools and application programming interfaces (APIs), execute code, and coordinate with other agents. This shift from a single model to an orchestrated software system dramatically enlarges the attack surface and introduces failure modes that classical machine-learning security and conventional application security do not fully capture. This paper presents a systematic literature review of security threats, attack surfaces, evaluation benchmarks, and mitigation strategies for agentic AI and LLM-based applications, covering the period January 2021 to June 2026 and reported in accordance with the PRISMA 2020 and PRISMA-S guidelines. A structured multi-database search returned 10,121 records; after duplicate removal, 6,221 unique records were screened, a prioritised candidate pool of 1,000 reports was retrieved, and 35 studies were retained for in-depth qualitative synthesis. We organise the evidence around five questions: the principal attack surfaces of agentic and LLM-based systems; the threats and vulnerabilities reported against them; the mitigation strategies proposed to counter those threats; the datasets, benchmarks, red-teaming methods and metrics used to evaluate security; and the open research gaps. The synthesis shows that prompt injection (direct and indirect), jailbreaks, sensitive-information disclosure, data and model poisoning, vector-database and retrieval poisoning, tool-use abuse, excessive agency, system-prompt leakage, insecure output handling, memory poisoning and AI supply-chain compromise dominate the reported threat landscape, and that these risks are amplified by autonomy, tool integration and multi-agent orchestration. Reported defences cluster into layered, defence-in-depth strategies—input and prompt hygiene, structured-query and alignment-based hardening, least-privilege and attribute-based access control, sandboxing and isolation, runtime monitoring with attacker-aware metrics, and governance frameworks aligned to OWASP, NIST, MITRE ATLAS and emerging regulation. We find that evaluation remains fragmented: Attack Success Rate is the dominant metric, benchmarks rarely test composed or chained cross-domain attacks, and defence evidence at the multi-agent and lifecycle level is comparatively weak. We conclude with a prioritised agenda for securing deployed agentic AI, including standardised attacker-aware benchmarks, continuous CI-integrated red-teaming, and defences that disrupt attack *progression* rather than hardening individual components in isolation.

**Keywords:** large language models; agentic AI; LLM agents; AI security; prompt injection; jailbreak; retrieval-augmented generation; tool use; excessive agency; red-teaming; systematic

literature review.

## Contents

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                     | <b>4</b>  |
| 1.1      | Scope and research questions . . . . .                                  | 4         |
| 1.2      | Contributions . . . . .                                                 | 5         |
| 1.3      | Paper organisation . . . . .                                            | 5         |
| <b>2</b> | <b>Background: The Anatomy of LLM-based and Agentic Systems</b>         | <b>6</b>  |
| <b>3</b> | <b>Review Methodology</b>                                               | <b>7</b>  |
| 3.1      | Protocol and reporting standard . . . . .                               | 7         |
| 3.2      | Information sources . . . . .                                           | 7         |
| 3.3      | Search strategy . . . . .                                               | 8         |
| 3.4      | Eligibility criteria . . . . .                                          | 8         |
| 3.5      | Selection and relevance prioritisation . . . . .                        | 9         |
| 3.6      | Full-text retrieval and re-ranking . . . . .                            | 9         |
| 3.7      | Data extraction and synthesis . . . . .                                 | 9         |
| 3.8      | Selection flow . . . . .                                                | 9         |
| <b>4</b> | <b>Search and Selection Results</b>                                     | <b>10</b> |
| 4.1      | Search yield and deduplication . . . . .                                | 10        |
| 4.2      | Database and query-origin contribution . . . . .                        | 10        |
| 4.3      | Temporal distribution and metadata completeness . . . . .               | 12        |
| 4.4      | Relevance prioritisation and full-text availability . . . . .           | 12        |
| 4.5      | Characteristics of the included studies . . . . .                       | 13        |
| <b>5</b> | <b>RQ1: Attack Surfaces in Agentic AI and LLM-based Applications</b>    | <b>13</b> |
| 5.1      | The prompt channel . . . . .                                            | 13        |
| 5.2      | Memory and persistent state . . . . .                                   | 13        |
| 5.3      | Retrieval and the vector store . . . . .                                | 14        |
| 5.4      | Tools, APIs and code execution . . . . .                                | 14        |
| 5.5      | Multi-agent orchestration and protocols . . . . .                       | 14        |
| 5.6      | The AI supply chain . . . . .                                           | 14        |
| <b>6</b> | <b>RQ2: Threats and Vulnerabilities</b>                                 | <b>14</b> |
| 6.1      | Prompt injection: direct and indirect . . . . .                         | 15        |
| 6.2      | Jailbreaks and safety bypass . . . . .                                  | 16        |
| 6.3      | Sensitive-information disclosure and data leakage . . . . .             | 16        |
| 6.4      | Poisoning: data, model, memory and retrieval . . . . .                  | 16        |
| 6.5      | Tool-use abuse, insecure output handling and excessive agency . . . . . | 16        |
| 6.6      | Multi-agent propagation and emergent risk . . . . .                     | 17        |
| 6.7      | AI supply-chain compromise . . . . .                                    | 17        |
| 6.8      | Composed and chained attacks . . . . .                                  | 17        |

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| <b>7</b>  | <b>RQ3: Mitigation Strategies and Defences</b>                       | <b>17</b> |
| 7.1       | Input, prompt and retrieval hygiene . . . . .                        | 17        |
| 7.2       | Model-level alignment and robustness . . . . .                       | 18        |
| 7.3       | Tool, execution and access control . . . . .                         | 18        |
| 7.4       | Orchestration- and lifecycle-level defences . . . . .                | 18        |
| 7.5       | Governance, standards and trust management . . . . .                 | 19        |
| 7.6       | Continuous, attacker-aware evaluation as a defence . . . . .         | 19        |
| <b>8</b>  | <b>RQ4: Datasets, Benchmarks, Red-teaming and Evaluation Metrics</b> | <b>19</b> |
| 8.1       | Benchmarks and datasets . . . . .                                    | 20        |
| 8.2       | Red-teaming methods . . . . .                                        | 20        |
| 8.3       | Evaluation metrics . . . . .                                         | 21        |
| 8.4       | Quantitative attack-success evidence . . . . .                       | 21        |
| <b>9</b>  | <b>RQ5: Research Gaps and Future Directions</b>                      | <b>21</b> |
| <b>10</b> | <b>Discussion</b>                                                    | <b>23</b> |
| <b>11</b> | <b>Threats to Validity</b>                                           | <b>23</b> |
| <b>12</b> | <b>Conclusion</b>                                                    | <b>24</b> |

# 1 Introduction

The release of instruction-tuned large language models (LLMs) triggered a rapid transition from chat assistants that merely produce text to *agentic* systems that act in the world. A modern LLM-based application seldom consists of a model in isolation: it is a software system in which the model is wrapped by prompts and system instructions, augmented with short- and long-term memory, connected to external knowledge through retrieval-augmented generation (RAG), and granted the ability to call tools, plugins and APIs, to execute code, and to delegate work to other agents in a multi-agent workflow [1–3]. This composition is precisely what makes such systems useful—and precisely what makes them dangerous. Each added capability is also an added entry point: a retrieved document can carry an injected instruction, a tool schema can be abused, a memory store can be poisoned, and an over-privileged action can turn a language error into a real-world consequence [4–6].

The pace of adoption has outstripped the maturity of the defences. The research literature on LLM-based agents has grown explosively—one synthesis reports that multi-agent-systems papers on a single preprint server rose from roughly 890 in 2019 to about 18,500 in 2024, and that LLM-based agent papers surged from near zero to several thousand following the public release of conversational models [2, 7]. At the same time, the threat surface has visibly migrated from theoretical concern to disclosed vulnerability: studies in our corpus catalogue remote-code-execution issues across more than a dozen agent frameworks, CVEs with critical CVSS scores in deployed agent tooling, and exposed agent endpoints reachable from the public internet [1, 8, 9]. Industry guidance has responded with dedicated taxonomies—the OWASP Top 10 for LLM Applications, the NIST AI Risk Management Framework, and MITRE ATLAS—but these are evolving in parallel with the systems they describe, and academic coverage of how they map onto agentic deployments is uneven [10, 11].

Existing surveys tend to address one slice of the problem at a time: a single threat class such as prompt injection [12, 13], a single architectural paradigm such as tool-calling versus the Model Context Protocol (MCP) [14], a governance perspective on enterprise adoption [11], or the use of LLMs as a *defensive* tool rather than as an asset that must itself be defended [15]. What is comparatively scarce is a consolidated view that follows the threat from the architecture that creates it, through the attacks that exploit it, to the benchmarks that measure it and the defences that contain it. This review aims to provide that consolidated view for agentic AI and LLM-based applications.

## 1.1 Scope and research questions

We scope the review to studies that address the cybersecurity of LLM-based and agentic AI systems themselves—their prompts, memory, RAG and vector stores, tools, APIs, plugins, code-execution environments and multi-agent workflows—rather than to studies that use AI only as a defensive instrument without considering the security of the AI system. Within this scope we organise the synthesis around five research questions (RQs):

**RQ1.**

What are the main attack surfaces in agentic AI and LLM-based applications?

**RQ2.**

What security threats and vulnerabilities have been reported for LLM-based agents, RAG systems, tool-use systems and autonomous AI workflows?

**RQ3.**

What mitigation strategies have been proposed to defend against prompt injection, jailbreaks, data leakage, poisoning, tool abuse and excessive agency?

**RQ4.**

What datasets, benchmarks, red-teaming methods and evaluation metrics are used to assess the security of LLM-based and agentic AI systems?

**RQ5.**

What are the major research gaps and future directions for the secure deployment of agentic AI systems?

## 1.2 Contributions

This review makes the following contributions:

- (i) A transparent, PRISMA-2020/PRISMA-S-compliant review process spanning twelve information sources and the January 2021–June 2026 window, with a fully documented search, selection and prioritisation pipeline and a PRISMA flow diagram (Section 3, Figure 1).
- (ii) A consolidated map of the agentic attack surface that links each architectural component (prompts, memory, RAG, tools/APIs, code execution and multi-agent orchestration) to the threats it enables (Section 5).
- (iii) A structured taxonomy of reported threats and vulnerabilities, with their relative prevalence in the synthesised corpus (Section 6).
- (iv) A synthesis of proposed mitigations organised as a defence-in-depth stack, together with the governance frameworks they align to (Section 7).
- (v) A consolidated account of benchmarks, red-teaming methods and evaluation metrics, including the quantitative attack-success evidence reported across studies (Section 8).
- (vi) A prioritised research agenda that identifies where the evidence is strongest, where it is weakest, and what must be measured next (Section 9).

## 1.3 Paper organisation

Section 2 describes the anatomy of LLM-based and agentic systems and the security-relevant properties of each component. Section 3 presents the review methodology, including the search strategy, eligibility criteria, selection and relevance prioritisation, full-text retrieval and evidence synthesis, and the PRISMA flow. Section 4 reports the search and selection results and the characteristics of the included studies. Sections 5–9 answer the five research questions in turn. Section 10 discusses cross-cutting findings, Section 11 addresses threats to the validity of the review, and Section 12 concludes.

## 2 Background: The Anatomy of LLM-based and Agentic Systems

To reason about the security of agentic AI it is necessary to be precise about what such a system is. An *LLM-based application* embeds a language model inside a larger software system that supplies it with instructions, context and the means to act. An *agentic* system goes further: it grants the model a degree of *autonomy*—the ability to decide, over multiple steps, which actions to take in pursuit of a goal, often using tools and external information, and sometimes in coordination with other agents [1, 3, 16]. This autonomy is the property that most distinguishes agentic security from ordinary model security: a wrong token in a chat reply is a nuisance, but a wrong *action* taken with real privileges—deleting data, transferring funds, executing code—is a security event.

The reviewed studies converge on a small number of architectural components, each of which is a security-relevant boundary. We summarise them below; their security implications are developed in Section 5.

**Prompts and system instructions.** The prompt is the control channel of an LLM. It combines a developer-authored *system prompt* (defining role, policy and tool permissions) with user input and, increasingly, third-party content. Because instructions and data share the same channel, a model can be induced to follow instructions that arrive inside what was supposed to be data—the root cause of prompt injection [12, 17]. The system prompt is itself a sensitive asset whose leakage exposes policy, tool schemas and guardrail logic [11].

**Memory and context.** Agents maintain short-term context (the conversation, intermediate results, scratchpads) and long-term memory (persisted facts, embeddings, episodic traces). Memory enables coherence across steps, but it also creates persistence: a malicious instruction or poisoned fact written to memory can influence future decisions long after the original interaction, a pattern the corpus describes as *memory poisoning* [2, 4].

**Retrieval-augmented generation (RAG) and vector databases.** RAG grounds an LLM in external documents retrieved from a vector store. It is widely treated as a reliability feature, but several studies stress that RAG is *not* intrinsically safer: retrieval introduces new failure modes, and index or document poisoning can yield high-success, targeted manipulation of the model’s output [6, 18]. Retrieved content is, from the model’s perspective, untrusted input that the model frequently fails to separate cleanly from its instructions.

**Tools, APIs, plugins and code execution.** Tool use is what turns a language model into an agent. The model emits a structured call—a tool name and arguments—which the runtime executes against an API, plugin, database or shell. This is the most consequential boundary in the stack: prompt-level manipulation translates directly into code execution or unsafe tool calls when argument handling, schemas or permissions are weak. One systematisation argues bluntly that an LLM with tools should be assumed to carry remote-code-execution risk unless proven otherwise [1, 5].

**Multi-agent orchestration.** In multi-agent systems, several LLM agents communicate, delegate and critique one another through inter-agent messages and shared state. Orchestration amplifies both capability and risk: a malicious message can propagate across agents, emergent and opaque behaviours can produce cascading errors, and trust must be managed across identity, planning, communication, memory, retrieval, execution and oversight boundaries [16, 19, 20].

**Communication protocols.** A newer layer of the stack standardises how agents reach tools and one another. Protocols such as the Model Context Protocol (MCP) and emerging agent-to-agent schemes define how capabilities are advertised and invoked; they are convenient, but they also become an attack surface in their own right, with reported protocol-level threats and exposed servers [8, 14, 20].

Taken together, these components define an expanded *trusted computing base* whose boundaries—between instructions and data, between the model and its tools, and between cooperating agents—are exactly where the reviewed threats concentrate.

## 3 Review Methodology

### 3.1 Protocol and reporting standard

The review was conducted and reported in accordance with the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA 2020) statement and the PRISMA-S extension for reporting literature searches. The review type is a systematic literature review. The search was last executed in June 2026 and covered publications dated between January 2021 and June 2026, restricted to English-language records. The objective, research questions (Section 1.1), eligibility criteria (Section 3.4) and information sources were defined in advance in a review configuration that fixed the topic definition, core concepts, search vocabulary and screening terms before retrieval.

### 3.2 Information sources

Twelve information sources were consulted: Scopus, Web of Science, IEEE Xplore, the ACM Digital Library, SpringerLink, ScienceDirect, PubMed, arXiv, Semantic Scholar, OpenAlex, Crossref and Europe PMC. Bibliographic records were retrieved programmatically from five of these sources (Crossref, OpenAlex, Europe PMC, PubMed and Semantic Scholar), which provide structured, query-addressable access to the indexed literature and, between them, cover the publishers and preprint servers relevant to this topic. To complement the peer-reviewed and preprint literature we also consulted authoritative grey-literature and standards sources, including the OWASP GenAI Security Project, the NIST AI Risk Management Framework, CISA AI security guidance, vendor red-team and threat reporting, and the proceedings of major security venues (USENIX Security, NDSS, IEEE S&P, ACM CCS, Black Hat and DEF CON); these informed the framing and the mapping of academic findings onto practitioner taxonomies.

### 3.3 Search strategy

The search strategy combined four complementary families of queries, all derived from the pre-specified review vocabulary:

- (1) **Full Boolean PRISMA-S search strings**—precision-oriented Boolean expressions combining population terms (e.g. “large language model”, “LLM”, “generative AI”, “agentic AI”, “AI agent”, “LLM agent”) with security terms (e.g. “security”, “attack”, “threat”, “vulnerability”, “prompt injection”, “jailbreak”, “mitigation”) and component terms (e.g. “retrieval augmented generation”, “RAG”, “vector database”, “tool use”, “code generation”);
- (2) **Database-specific PRISMA-S queries**—the same intent rendered to the syntax and field constraints of each source’s interface;
- (3) **Generated topic queries**—compact phrase queries expressing the review intent (e.g. “LLM agent security systematic review”, “agentic AI cybersecurity threats mitigation benchmark”, “retrieval augmented generation security attacks mitigation”);
- (4) **Keyword and free-text queries**—short controlled-vocabulary and free-text terms used as broad-recall probes.

Each query was expanded across the queryable databases, yielding 150 query–database runs, of which 119 (79.3%) were productive (returned at least one record), 7 (4.7%) returned no records, and 24 (16.0%) failed at the interface level. All runs were restricted to the 2021–2026 window. The full set of executed query–database runs, with their origin, rendered form and returned counts, is documented in the search log accompanying this review.

### 3.4 Eligibility criteria

**Inclusion criteria.** A record was eligible if it (i) addressed cybersecurity threats, vulnerabilities, attacks or defences involving large language models, generative-AI systems or agentic-AI systems; (ii) concerned LLM-based applications with components such as prompts, memory, RAG, vector databases, APIs, plugins, tools, code execution or autonomous workflows; (iii) proposed, evaluated or benchmarked attacks, mitigations, red-teaming methods or security-evaluation frameworks for such systems; (iv) was a peer-reviewed article, conference paper, preprint, technical report or high-quality security-framework document relevant to LLM and agentic-AI security; and (v) was written in English.

**Exclusion criteria.** A record was excluded if it (i) discussed AI in cybersecurity only as a defensive tool without addressing the security of the AI or LLM-based system itself; (ii) focused only on traditional machine-learning security without relevance to LLMs, generative AI or agentic AI; (iii) addressed general AI ethics, policy or governance without technical cybersecurity content; (iv) was an editorial, opinion piece, blog post, news article or non-technical commentary without a clear method, taxonomy, experiment, framework or security analysis; or (v) was a duplicate record, an inaccessible full text, or a non-English study.

### 3.5 Selection and relevance prioritisation

After retrieval and metadata cleaning, records were deduplicated using the Digital Object Identifier (DOI) where available, followed by normalised title matching and a robust identifier hierarchy (DOI, OpenAlex ID, PubMed ID, Semantic Scholar ID, and normalised title plus publication year). The deduplicated records were then screened for relevance to the review objective. Because the unique pool was large (several thousand records), title/abstract screening was supported by two complementary relevance models that represented the review intent and compared it against each record’s title and abstract: a focused representation built from the review title and objective, and a broader representation that additionally incorporated the research questions, core concepts, search queries, keywords and eligibility terms. The two configurations were used as cross-checks on one another; their agreement on the prioritised pool was high (87.4% overlap at the top 1,000 records, and 79% at the top 100), which supports the stability of the ranking (Section 4). The most relevant 1,000 records were retained as the candidate pool for retrieval.

### 3.6 Full-text retrieval and re-ranking

For the candidate pool, full texts were sought and, where retrievable, extracted. A retrieved document was treated as usable when a valid PDF was present and machine-readable text could be extracted. The candidate pool was then re-ranked using full-text-aware relevance models that combined lexical matching with semantic similarity over scientific-document representations, fused into a single hybrid ordering; records without extractable full text were retained but ranked on title, abstract and metadata. The most relevant 200 full-text-ranked reports were carried forward as the assessment pool, and the 50 highest-ranked reports were assessed in depth for eligibility and evidence extraction.

### 3.7 Data extraction and synthesis

For each report assessed in depth, evidence was extracted into a structured article profile covering: the screening decision and its justification; article type, aim and study domain; the research questions addressed; datasets and data sources; sample size or scale; attack types and threats; systems, tools and frameworks; methods, models and algorithms; evaluation design, metrics and numerical results; reported novelty, strengths and weaknesses; limitations and future work; quality and risk-of-bias notes; and the tables, figures and taxonomies presented. Extraction used only the supplied review context and the report’s own text; unsupported claims were avoided, and missing information was recorded as not reported rather than inferred. The extracted profiles were then synthesised thematically across reports to answer the research questions; the prevalence figures reported in Sections 6–8 are article-level counts in which each report contributes at most once to a given concept label. Extracted numerical results, dataset names and limitation statements were audited against the source reports before being used in manuscript claims.

### 3.8 Selection flow

Figure 1 summarises the selection flow. The database searches returned 10,121 records; removing duplicates and cross-database overlap left 6,221 unique records for screening. Relevance prioritisa-

tion excluded 5,221 records below the candidate threshold and retained 1,000 reports for retrieval. Of these, 301 could not be retrieved with accessible full text. The retrieved reports were prioritised by full-text relevance (top 200), and the 50 highest were assessed in depth; 15 were excluded—8 as duplicate or alternate versions of a retained report (for example, a preprint and its journal version), 6 for insufficient extractable full text, and 1 for not meeting the inclusion criteria on full-text review—leaving 35 studies included in the qualitative synthesis. The broader top-200 full-text-ranked pool underpins the quantitative search-and-selection reporting in Section 4, while the 35 included studies form the corpus for the thematic synthesis.

## 4 Search and Selection Results

### 4.1 Search yield and deduplication

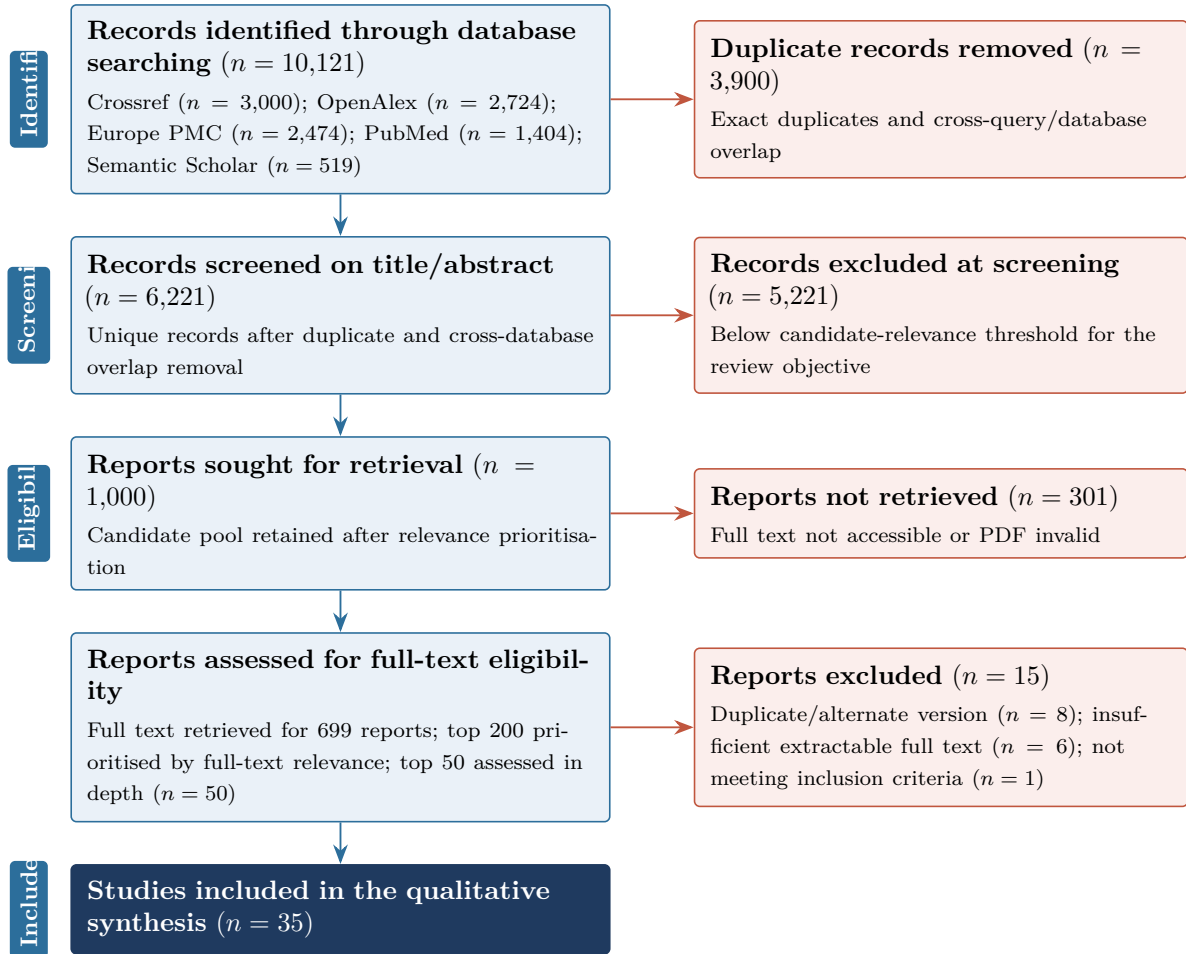
The search returned 10,121 records across the queryable databases. After deduplication, 6,221 unique records were retained—an estimated duplicate and overlap reduction of roughly 39%, reflecting both exact duplicates and the substantial overlap expected when the same article is indexed by several databases. DOIs were present for 6,066 records (97.5%), which supported reliable deduplication and downstream linkage. Table 1 summarises the headline search figures, and Figure 2 presents the search dashboard, including the contribution of each database, the distribution of retained records by year, and the completeness of key metadata fields.

**Table 1:** Overall search and deduplication summary.

| Metric                                       | Value         |
|----------------------------------------------|---------------|
| Query-database runs                          | 150           |
| Productive runs                              | 119 (79.3%)   |
| Zero-result runs                             | 7 (4.7%)      |
| Failed runs                                  | 24 (16.0%)    |
| Total records returned (pre-deduplication)   | 10,121        |
| Unique records retained (post-deduplication) | 6,221         |
| Estimated duplicate/overlap reduction        | ≈ 39%         |
| Records with DOI                             | 6,066 (97.5%) |
| Publication-year range                       | 2021–2026     |

### 4.2 Database and query-origin contribution

Among the retained records, source-database contribution was led by Crossref (1,987 records; 31.9%), followed by OpenAlex (1,726; 27.7%), Europe PMC (1,560; 25.1%), PubMed (1,010; 16.2%) and Semantic Scholar (509; 8.2%); because a deduplicated record may be linked to more than one source, these shares reflect overlapping credit rather than a partition. On the query side, broad keyword and generated topic queries provided the strongest recall, while the Boolean PRISMA-S strings provided precision support; PubMed and Semantic Scholar showed lower productive-run rates, the latter largely because of interface-level run failures rather than empty results. Table 2 reports the database contribution and productivity.



**Figure 1:** PRISMA 2020 flow of records through identification, screening, eligibility and inclusion. Database searching returned 10,121 records; 3,900 duplicates were removed; 6,221 unique records were screened; 1,000 reports were sought for retrieval; 301 could not be retrieved; and after full-text eligibility assessment, 35 studies were included in the qualitative synthesis. The top-200 full-text-ranked pool supports the quantitative search-and-selection reporting in Section 4.

**Table 2:** Source-database contribution and productivity among retained records. Unique-record credit can overlap across databases.

| Database         | Unique credit | % retained | Productive runs |
|------------------|---------------|------------|-----------------|
| Crossref         | 1,987         | 31.9%      | 100.0%          |
| OpenAlex         | 1,726         | 27.7%      | 100.0%          |
| Europe PMC       | 1,560         | 25.1%      | 100.0%          |
| PubMed           | 1,010         | 16.2%      | 76.7%           |
| Semantic Scholar | 509           | 8.2%       | 20.0%           |

### 4.3 Temporal distribution and metadata completeness

The retained corpus is strongly skewed toward recent work, consistent with the rapid emergence of the field: 121 records were dated 2021, rising to 518 in 2023, 1,082 in 2024, 1,988 in 2025 and 2,327 in the partial year 2026 (Table 3). More than two-thirds of the retained records (and the overwhelming majority of the included studies) were published in 2024 or later. Metadata completeness was high for titles (100%), URLs (100%), years (99.7%), DOIs (97.5%) and authors (96.5%), and more moderate for abstracts (63.8%) and journal/venue fields (65.6%), the latter reflecting the large share of preprints and recently indexed records.

**Table 3:** Year distribution of retained records (2026 is a partial year through June).

| Year    | 2021 | 2022 | 2023 | 2024  | 2025  | 2026  | Missing |
|---------|------|------|------|-------|-------|-------|---------|
| Records | 121  | 168  | 518  | 1,082 | 1,988 | 2,327 | 17      |

### 4.4 Relevance prioritisation and full-text availability

Relevance prioritisation reduced the 6,221 unique records to a 1,000-report candidate pool. The two prioritisation configurations agreed on 87.4% of the top-1,000 pool, and the mean relevance score declined monotonically across successive rank bands, indicating that the configured topic and security vocabulary remained concentrated in the highest-ranked records rather than being diffused through the pool (Figure 3). Validation terms drawn from the review configuration—“large language model”, “security”, “LLM”, “agent”, “prompt”, “attack” and related terms—were present in the expected high proportions among top-ranked records, and every top-ranked record contained at least one configured validation term.

Full texts were then sought for the candidate pool. Of the 1,000 candidate reports, 699 (69.9%) yielded a valid, machine-readable PDF and 561 (56.1%) yielded usable extracted text, together representing approximately 5.9 million words (median 7,882 words per article). The full-text-aware re-ranking carried the top 200 reports forward, of which 192 (96.0%) had extractable full text—a deliberately higher full-text rate than the candidate pool as a whole, because the re-ranking favoured reports with substantive, analysable content (Figure 4). Open-access discovery routes dominated retrieval, with OpenAlex the most frequent originating database among full-text-available reports.

## 4.5 Characteristics of the included studies

Thirty-five studies were included in the qualitative synthesis. They are predominantly recent (2024–2026) and span peer-reviewed journal articles, conference papers, preprints, technical reports and security-framework documents, consistent with a fast-moving field in which preprints and standards bodies carry a large share of the timely evidence. By article type, the corpus is dominated by surveys, systematisations of knowledge (SoKs) and taxonomy/threat-model papers, complemented by a smaller set of empirical evaluations and benchmark contributions. Thematically, every included study addressed at least one review question, and coverage of the major synthesis themes—review-question coverage, threat/attack focus, methods and models, main findings, and limitations—was effectively complete across the corpus (Figure 5). The most frequently addressed research questions concerned attack surfaces (addressed by 20 of 36 profiled reports), mitigation strategies (19), and reported threats and vulnerabilities (18), with research gaps (15) and benchmarks/metrics (13) also well represented. Figure 6 summarises the most frequent extracted concepts by category, which we draw on directly in Sections 6–8.

## 5 RQ1: Attack Surfaces in Agentic AI and LLM-based Applications

The reviewed studies are consistent on a central point: the attack surface of an agentic system is not the model, but the *system around the model*. Several systematisations frame this explicitly in terms of an expanded trusted computing base whose boundaries—between instructions and data, between the agent and its tools, and between cooperating agents—are where exploitation concentrates [1, 3]. We organise the attack surface by architectural component, following the anatomy of Section 2, and summarise the component-to-threat mapping in Table 4.

### 5.1 The prompt channel

The most fundamental surface is the prompt itself. Because the system prompt, user input and any third-party content share a single, unstructured channel, an agent cannot reliably tell an instruction from data. This is the structural cause of prompt injection, and it is why studies repeatedly find that models “fail to reliably separate retrieved content from instructions” [6, 12]. The system prompt is also an asset in its own right: its leakage discloses role definitions, tool schemas, policy and guardrail logic, and is recorded as a distinct threat (*system-prompt leakage*) across the corpus [11, 21].

### 5.2 Memory and persistent state

Memory turns a transient manipulation into a durable one. An instruction or fact written to short- or long-term memory can shape later decisions, enabling delayed-effect attacks and goal drift that are difficult to detect because cause and effect are separated in time [2, 4]. Memory poisoning is reported as a recurring threat (Section 6) and is one of the few attack classes that is essentially unique to stateful agents.

### 5.3 Retrieval and the vector store

RAG widens the surface by admitting external, untrusted documents into the model’s context. The corpus is emphatic that retrieval is not a safety feature: poisoning the index or the retrieved documents yields high-success, targeted manipulation, and retrieval introduces failure modes of its own [6, 18]. Vector-database attacks—poisoning embeddings, manipulating similarity, or injecting adversarial passages—are reported as a distinct and prevalent threat.

### 5.4 Tools, APIs and code execution

Tool use is the highest-consequence surface because it is the bridge from language to action. When argument handling, tool schemas or permissions are weak, prompt-level manipulation translates directly into unsafe tool calls or code execution. One systematisation documents remote-code-execution issues across eleven agent frameworks and argues that an LLM granted tools should be treated as carrying RCE risk unless proven otherwise [1]; a complementary analysis of agentic coding assistants reports attack success rates above 85% against state-of-the-art defences once injection reaches the tool layer [5]. *Excessive agency*—granting an agent more capability, autonomy or permission than its task requires—is repeatedly identified as the property that converts a model error into a security incident [8, 21].

### 5.5 Multi-agent orchestration and protocols

In multi-agent systems the surface includes the messages and shared state through which agents coordinate. A malicious message can propagate across agents, and early test-beds demonstrate cross-agent propagation and emergent, opaque behaviours that produce cascading errors [16, 19]. The communication protocols that standardise agent-to-tool and agent-to-agent interaction—most prominently MCP—are themselves an attack surface, with reported protocol-level threats and publicly exposed servers [8, 14, 20]. A comparative study of deployment paradigms found that the tool-calling and MCP architectures present materially different vulnerability profiles, underscoring that the *choice of integration layer* is itself a security decision [14, 22].

### 5.6 The AI supply chain

Finally, the corpus treats the AI supply chain as a first-class surface. Models, adapters, datasets, embeddings, plugins and community-contributed tools are all dependencies that can be compromised upstream. AI supply-chain vulnerabilities were the single most frequently reported threat category in the synthesis (Section 6); one case study of an autonomous-agent ecosystem found that roughly a quarter of community-contributed tools contained security issues [9, 23].

## 6 RQ2: Threats and Vulnerabilities

Across the synthesised corpus, threat and attack content was effectively universal: every profiled report discussed at least one attack type, threat or risk category. Table 5 ranks the most frequently reported threat categories by the share of studies that discuss them. The distribution confirms the picture from RQ1: the leading risks are those introduced or amplified by autonomy, tool

**Table 4:** Mapping of architectural components to the principal attack surfaces and representative threats reported across the corpus.

| Component                               | What it exposes                                         | Representative threats                                                            |
|-----------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Prompt channel</b>                   | Shared instruction/data channel; system prompt as asset | Direct/indirect prompt injection; jailbreaks; system-prompt leakage               |
| <b>Memory &amp; state</b>               | Persistence of instructions and facts across steps      | Memory poisoning; delayed-effect manipulation; goal drift                         |
| <b>RAG &amp; vector store</b>           | Admission of untrusted external documents               | Retrieval/index poisoning; vector-database attacks; data leakage                  |
| <b>Tools, APIs &amp; code execution</b> | Bridge from language to privileged action               | Tool-use abuse; insecure output handling; remote code execution; excessive agency |
| <b>Multi-agent &amp; protocols</b>      | Inter-agent messages, shared state, integration layer   | Cross-agent propagation; collusion; protocol-level threats; emergent misbehaviour |
| <b>AI supply chain</b>                  | Upstream models, plugins, tools, data                   | Poisoned dependencies; backdoors; compromised community tools                     |

integration, retrieval and orchestration, rather than the text-only failure modes familiar from standalone models.

**Table 5:** Most frequently reported threat and vulnerability categories across the synthesised studies (article-level prevalence; categories with closely related labels merged).

| Threat / vulnerability category     | % of studies |
|-------------------------------------|--------------|
| AI supply-chain vulnerabilities     | 25.0%        |
| Excessive agency                    | ≈ 14%        |
| Insecure output handling            | ≈ 14%        |
| System-prompt leakage               | ≈ 14%        |
| Indirect prompt injection           | ≈ 14%        |
| Prompt injection (direct)           | ≈ 11%        |
| Jailbreaks                          | ≈ 11%        |
| Sensitive-information disclosure    | ≈ 11%        |
| Vector-database / retrieval attacks | ≈ 11%        |
| Tool-use abuse                      | ≈ 11%        |
| Memory poisoning                    | ≈ 11%        |
| Data and model poisoning            | ≈ 8%         |

## 6.1 Prompt injection: direct and indirect

Prompt injection is the most thoroughly studied threat in the corpus and the one with the clearest structural cause. In *direct* injection, a user crafts input that overrides the system prompt or policy; in *indirect* injection, the malicious instruction is embedded in content the agent ingests—a retrieved document, a web page, an email, a tool result—and is executed when the agent processes it [12, 17]. Indirect injection is repeatedly described as both highly practical and difficult to eliminate, precisely because models do not cleanly separate retrieved content from instructions [6]. Reported pre-mitigation success rates are high: one study measured a direct-prompt-injection success rate of 82% before mitigation [17], and analyses of agentic coding assistants report injection success above 85% against current defences once the payload reaches the tool or protocol layer

[5]. A dedicated systematic review of prompt injection consolidates the taxonomy and traces its evolution from simple overrides to optimisation-based and cross-domain variants [12, 13].

## 6.2 Jailbreaks and safety bypass

Jailbreaks—inputs that bypass alignment and safety policy to elicit prohibited behaviour—remain prevalent and overlap with injection in agentic settings, where a successful bypass can be chained into a tool action rather than merely producing disallowed text. Reported jailbreak techniques in the corpus include role-playing, obfuscation and optimisation-based methods such as greedy coordinate-gradient search and automated adversarial generation [3, 6]. The key agentic concern is integrity subversion: a jailbreak that survives into the action layer becomes a safety bypass with real-world effect.

## 6.3 Sensitive-information disclosure and data leakage

Sensitive-information disclosure—of training data, system prompts, retrieved private documents, tool credentials or user data—is a consistently reported outcome, and is closely tied to RAG and memory. Enterprise-focused reviews place data leakage and prompt injection among the most-cited risks across their source bases [11, 24]. In multi-tenant deployments, the disclosure risk is amplified: a systematic analysis of LLM-powered multi-tenant SaaS found that a substantial fraction of identified vulnerabilities were worsened by multi-tenancy relative to single-tenant deployment [24].

## 6.4 Poisoning: data, model, memory and retrieval

Poisoning attacks appear in four guises in the corpus—training-data poisoning, model/backdoor poisoning, memory poisoning, and retrieval/index poisoning—and a dedicated survey treats adversarial, poisoning and backdoor attacks on large model-based agents as a unified family [6]. Retrieval poisoning is singled out as especially practical because it does not require access to model weights: inserting crafted documents into a knowledge source can steer outputs with high targeted success [18]. Memory poisoning extends this into persistence, allowing an attacker’s influence to outlive the originating interaction [4].

## 6.5 Tool-use abuse, insecure output handling and excessive agency

Three tightly linked threats dominate the action layer. *Tool-use abuse* occurs when an agent is induced to call tools in unintended ways—exfiltrating data, invoking destructive operations, or escalating privilege. *Insecure output handling* occurs when an agent’s output (including tool arguments or generated code) is consumed downstream without validation, turning a generation error into an injection or execution vulnerability. *Excessive agency* is the amplifier: an over-permissioned agent converts either of the former into a high-impact event [1, 21]. Threat models in the corpus formalise these into structured catalogues—for example, a threat model that identifies nine primary threats across five domains (cognitive-architecture, temporal-persistence, operational-execution, trust-boundary and supply-chain) for generative-AI agents [21].

## 6.6 Multi-agent propagation and emergent risk

Multi-agent systems add collusion, cross-agent propagation and emergent misbehaviour to the threat list. Reviews of multi-agent and trustworthy-agent systems report that orchestration amplifies risk, that malicious messages propagate across agents in early test-beds, and that agentic threats exhibit delayed effects, goal-misalignment magnification and cross-system propagation that single-agent threat models do not capture [2, 16, 19]. These properties make detection and attribution harder, a point we return to under RQ4 and RQ5.

## 6.7 AI supply-chain compromise

The most frequently reported single category, AI supply-chain vulnerability, spans poisoned base models, compromised adapters and datasets, and insecure community-contributed plugins and tools. Case studies of autonomous-agent ecosystems make the risk concrete: one reports that approximately 26% of community-contributed tools contained security issues, and another documents a named exploit class and a critical CVE in a widely used autonomous agent [9, 23]. Surveys of trustworthy agentic AI similarly catalogue critical-severity CVEs and large numbers of exposed agent endpoints reachable from the public internet [8].

## 6.8 Composed and chained attacks

A recurring and important finding is that real-world risk is systematically *underestimated* by single-vector evaluation. A comparative vulnerability assessment of LLM-agent deployment paradigms reports that composed attacks—which link an AI vulnerability to a software vulnerability—achieve success rates far higher than isolated attacks, with chained attacks reaching 91–96% success against roughly 50% for simple single-vector attempts [14, 22]. The same work measures architecture-dependent exposure, finding an overall attack-success rate of 73.5% for a tool-calling (function-calling) architecture versus 62.59% for an MCP-based architecture, with the function-calling architecture especially exposed at the system level (87.98%) and the MCP architecture more exposed at the model level (68.28%). The practical implication, stated across several studies, is that security testing must include composed and chained scenarios, and that defences should target attack *progression* rather than hardening individual components in isolation.

# 7 RQ3: Mitigation Strategies and Defences

No single safeguard was found to be sufficient in any reviewed study; the consistent message is that layered, defence-in-depth strategies meaningfully reduce unsafe actions and limit blast radius, whereas point defences fail under composed attacks [1, 25]. We organise the reported mitigations as a stack, from the prompt and input layer through the model, tool and orchestration layers to governance, and summarise the mapping in Table 6.

## 7.1 Input, prompt and retrieval hygiene

The first layer addresses the prompt channel directly: prompt sanitisation, context isolation that separates instructions from data, input filtering and detection of adversarial prompts, and

validation of retrieved content before it enters the context. A multilayered mitigation framework for prompt injection reports substantial reductions in attack success when these controls are combined [17, 26]. A structured-query approach (StruQ), which forces a clean separation between the instruction and data channels, is reported to reduce attack success to below 2% for most evaluated attacks, and robustness-checking defences (e.g. RA-LLM) further reduce success [13]. An empirical multi-model study of layered controls reports detection-rate improvements from combining input filtering and prompt hardening—for example, raising detection on one model from 66% to 90%—while emphasising the security–usability trade-off that aggressive filtering introduces [25].

## 7.2 Model-level alignment and robustness

The second layer hardens the model. Reported techniques include adversarial training, alignment-based defences such as preference optimisation and secure-alignment fine-tuning, and reasoning-time defences (reflection, self-criticism, chain-of-thought verification, subgoal decomposition) used to make the model more resistant to manipulation [6, 21]. The evidence here is mixed: alignment improves average robustness but does not eliminate optimisation-based jailbreaks, and one cross-cutting finding is that more capable reasoning models can be *more* exploitable even as they detect threats better, so model capability is not a substitute for system-level controls [14].

## 7.3 Tool, execution and access control

The third layer constrains what an agent can actually do. Reported controls include least-privilege and attribute-based access control (ABAC) over tools and data, sandboxing and isolation of code execution, capability gating and human-in-the-loop approval for high-impact actions, and architectural isolation patterns such as separating a planning “action-selector” from execution and isolating untrusted plugins [8, 27]. Guardrail and policy systems—content classifiers applied to inputs and outputs, and policy enforcement at the tool boundary—are reported as effective complements, with authorization frameworks such as a context-aware verifiable policy applied to autonomous agents [27]. The recurring principle is that excessive agency is mitigated by reducing agency: scoping permissions to the task is the most direct control over the highest-consequence surface.

## 7.4 Orchestration- and lifecycle-level defences

The fourth layer addresses multi-agent and lifecycle threats. Proposed measures include zero-trust architectures for inter-agent communication, provenance and attestation of messages and tool calls, decision-provenance logging, and lifecycle-oriented security frameworks that span design-time hardening, runtime monitoring and incident response [19, 23, 28]. Unified threat-and-defence frameworks bring these together: a defence-in-depth framework paired with a phased security checklist (design-time, runtime, incident-response) and a mitigation framework (e.g. SHIELD) mapped to a structured threat model (e.g. ATFAA), with explicit mappings to OWASP GenAI and MITRE ATLAS [10, 21]. A unified security orchestration framework with multiple integrated modules reports an empirical prompt-attack detector (an XGBoost classifier over TF-IDF features) achieving high precision on a labelled interaction set [29].

## 7.5 Governance, standards and trust management

The top layer is organisational. Several studies map technical controls onto governance frameworks and regulation, arguing that secure agentic deployment is as much a governance problem as an engineering one. The TRiSM (Trust, Risk and Security Management) framework for LLM-based agentic multi-agent systems is the most developed example: it integrates explainability, model operations, security, privacy and lifecycle governance, proposes coordination- and tool-use-quality metrics (Component Synergy Score and Tool Utilization Efficacy), and maps its pillars to the EU AI Act, the NIST AI RMF and ISO/IEC standards [2, 7]. Enterprise reviews reach a similar conclusion, recommending that organisations prioritise AI governance, secure prompt engineering, continuous monitoring and alignment with OWASP/NIST baselines, and noting rising security-oversight budgets as adoption matures [11, 18].

## 7.6 Continuous, attacker-aware evaluation as a defence

Finally, the corpus treats measurement itself as a control. Because the threat landscape evolves quickly and adversaries adapt to static defences, several studies argue that security must be measured continuously with attacker-aware metrics and CI-integrated red-teaming, rather than certified once at release [1, 30]. Proposed operationalisations include attacker-aware metrics computed from agent traces (e.g. unauthorized-action and progression-related rates) that can be embedded in CI/CD pipelines for continuous security evaluation. This connects directly to RQ4.

**Table 6:** Defence-in-depth stack synthesised from the corpus: layers, representative controls, and the frameworks they align to.

| Layer                             | Representative controls                                                                                                               | Aligns to                             |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <b>Input / prompt / retrieval</b> | Prompt sanitisation; instruction–data separation (StruQ); input filtering; adversarial-prompt detection; retrieved-content validation | OWASP LLM Top 10                      |
| <b>Model</b>                      | Adversarial training; secure alignment / preference optimisation; reasoning-time verification (reflection, self-criticism)            | NIST AI RMF                           |
| <b>Tool / execution / access</b>  | Least-privilege & ABAC; sandboxing/isolation; capability gating; human-in-the-loop approval; guardrail classifiers                    | OWASP Agentic Top 10; MITRE ATLAS     |
| <b>Orchestration / lifecycle</b>  | Zero-trust inter-agent comms; provenance & attestation; lifecycle hardening; phased checklists (SHIELD/ATFAA)                         | MITRE ATLAS; CSA MAESTRO              |
| <b>Governance</b>                 | TRiSM; risk management; metrics (CSS, TUE); regulatory mapping                                                                        | EU AI Act; NIST AI RMF; ISO/IEC 42001 |
| <b>Continuous evaluation</b>      | Attacker-aware metrics; CI-integrated red-teaming; trace-based monitoring                                                             | OWASP; MITRE ATLAS                    |

## 8 RQ4: Datasets, Benchmarks, Red-teaming and Evaluation Metrics

Evaluation evidence was present in most reviewed studies, but it was fragmented: benchmarks proliferate, metrics are inconsistently defined, and few benchmarks test the composed and chained

attacks that the same literature identifies as most dangerous. This section consolidates the datasets and benchmarks, the red-teaming methods, the metrics, and the quantitative attack-success evidence.

## 8.1 Benchmarks and datasets

Table 7 lists the benchmarks and datasets most frequently referenced in the corpus. They fall into three groups: agentic-security benchmarks that evaluate attacks and defences against tool-using agents (Agent Security Bench, AgentDojo, InjecAgent, R-Judge); jailbreak and harmful-behaviour benchmarks (HarmBench, JailbreakBench, JAILBREAKHUB); and agent capability/task benchmarks repurposed for security evaluation (ToolBench, API-Bank, AgentBench, GAIA, WebArena, HELM), together with cyber-specific sets (CyberGym, ExCyTIn-Bench, CyberMetric).

**Table 7:** Benchmarks and datasets most frequently referenced across the synthesised studies, grouped by purpose.

| Group                                | Representative benchmarks/datasets                         |
|--------------------------------------|------------------------------------------------------------|
| <b>Agentic-security</b>              | Agent Security Bench (ASB); AgentDojo; InjecAgent; R-Judge |
| <b>Jailbreak / harmful behaviour</b> | HarmBench; JailbreakBench; JAILBREAKHUB                    |
| <b>Agent capability / task</b>       | ToolBench; API-Bank; AgentBench; GAIA; WebArena; HELM      |
| <b>Cyber-specific</b>                | CyberGym; ExCyTIn-Bench; CyberMetric                       |

The most detailed single benchmark contribution is Agent Security Bench (ASB), which formalises and evaluates multiple attack types—direct and indirect prompt injection, memory poisoning, a plan-of-thought backdoor and mixed attacks—and corresponding defences across a large matrix of scenarios, agents, tools and tasks (on the order of ten scenarios, ten agents, several hundred tools and tasks, and a dozen-plus model backbones), with simulated tool calls for reproducibility [4]. ASB reports that mixed attacks combining multiple vulnerabilities are the most effective, with the highest average attack-success rate among the evaluated attack types, and that current defences are only partially effective—motivating its proposed resilience-oriented metric that balances utility against security. A broad survey of agentic security covering hundreds of papers reaches a complementary conclusion: most attacks against agents are inherited or amplified from base-model attacks, and existing benchmarks under-represent agent-specific and cross-domain threats [31].

## 8.2 Red-teaming methods

Red-teaming in the corpus ranges from manual scenario-based testing to automated adversarial generation. Reported approaches include scenario-based red teaming, automated adversarial generation using generator models, lifecycle-based adversarial threat taxonomies, and LLM-driven adversarial testing in which one model attacks another [30, 32]. A practitioner-oriented contribution structures security testing into a phased, five-stage framework spanning reconnaissance to exploitation, mapped to OWASP LLM and agentic top-ten lists and MITRE ATLAS, illustrating the move toward repeatable, standards-aligned red-teaming [33]. Offensive-agent studies demonstrate the dual-use dimension concretely: an autonomous offensive agent built on a general-purpose

model and RAG was reported to compromise vulnerable targets at trivial cost, underscoring that the same agentic capabilities under study can themselves be weaponised [34, 35].

### 8.3 Evaluation metrics

Metric usage is concentrated but inconsistent. Attack Success Rate (ASR) is by far the dominant metric, reported by roughly a third of the synthesised studies, often without a shared definition of “success” across attack types. Around it sit False Positive Rate, Refusal Rate (RR), precision, recall, accuracy, and a set of bespoke composite indices proposed by individual frameworks—Component Synergy Score and Tool Utilization Efficacy for multi-agent coordination and tool use, robustness and defence-effectiveness scores, and resilience/composite metrics that attempt to trade off utility against security [2, 4]. Table 8 summarises the most frequent metrics. The proliferation of bespoke composites, while useful within each study, hampers cross-study comparison and is one of the clearest evaluation gaps (Section 9).

**Table 8:** Evaluation metrics most frequently reported across the synthesised studies.

| Metric                                                          | % of studies |
|-----------------------------------------------------------------|--------------|
| Attack Success Rate (ASR)                                       | ≈ 31%        |
| False Positive Rate                                             | ≈ 11%        |
| Refusal Rate (RR)                                               | ≈ 11%        |
| Precision                                                       | ≈ 8%         |
| Recall                                                          | ≈ 8%         |
| Accuracy                                                        | ≈ 8%         |
| Component Synergy Score (CSS) / Tool Utilization Efficacy (TUE) | ≈ 6%         |
| Robustness / Defence-effectiveness scores                       | ≈ 6%         |

### 8.4 Quantitative attack-success evidence

Despite the inconsistency in definitions, the reported numbers tell a coherent story, summarised in Table 9. Single-vector attacks succeed moderately often (around 50%), but composed and chained attacks succeed far more often (91–96%), and architecture choice shifts both the overall exposure and where it concentrates (function-calling versus MCP) [14, 22]. Model choice matters but does not confer safety: reported per-model ASRs cluster in the 65–75% range across leading models, and reasoning-capable models show low average refusal rates even as they detect more threats [14]. On the defensive side, instruction–data separation can drive injection success below 2%, and layered controls lift detection substantially, but these gains are demonstrated mostly in constrained settings rather than against composed, cross-domain attacks [13, 25]. In the application domain, capability benchmarks show strong but uneven model performance—for example, high accuracy on cyber-knowledge question sets that degrades on harder, larger sets—reinforcing that benchmark scores are context-dependent [36, 37].

## 9 RQ5: Research Gaps and Future Directions

The corpus is candid about its own limits, and the limitation statements extracted across studies converge on a small set of structural gaps. Almost every study notes that the security landscape evolves rapidly and that its findings are a temporal snapshot; many are theoretical or survey

**Table 9:** Representative quantitative results reported across the corpus. Values are as reported by the cited studies and use heterogeneous definitions; they are indicative rather than directly comparable.

| Result                                          | Context                                                                          | Source |
|-------------------------------------------------|----------------------------------------------------------------------------------|--------|
| Chained-attack ASR 91–96% vs. $\sim$ 50% simple | Composed vs. single-vector attacks                                               | [14]   |
| Function-calling overall ASR 73.5%; MCP 62.59%  | Architecture comparison; FC system-centric 87.98%, MCP LLM-centric 68.28%        | [22]   |
| Per-model ASR $\approx$ 65–75%                  | Leading models under adversarial testing; reasoning-model avg RR $\approx$ 17.8% | [14]   |
| Mixed-attack ASR highest among attack types     | ASB benchmark; defences only partially effective                                 | [4]    |
| Direct-injection success 82% pre-mitigation     | Multilayered mitigation framework                                                | [17]   |
| Injection success $<$ 2% post-defence (StruQ)   | Instruction–data separation defence                                              | [13]   |
| Detection lifted (e.g. 66% $\rightarrow$ 90%)   | Layered input filtering + prompt hardening                                       | [25]   |
| $\sim$ 26% of community tools insecure          | Autonomous-agent ecosystem case study                                            | [23]   |
| RCE across 11 agent frameworks                  | Attack-surface systematisation                                                   | [1]    |

contributions without empirical validation; and defence evidence is markedly weaker than threat evidence, particularly at the multi-agent and lifecycle level. We distil six priorities.

**1. Benchmarks for composed, chained and cross-domain attacks.** The single most consequential gap is the mismatch between how attacks succeed and how they are measured. The literature shows that composed and chained attacks dominate real-world risk, yet leading benchmarks (AgentDojo, InjecAgent, ASB) largely evaluate single-vector attacks and do not assess cross-domain or architectural security comprehensively [4, 14]. Standard benchmarks that compose AI and software vulnerabilities, and that test attack *progression*, are needed.

**2. Standardised, attacker-aware metrics.** ASR dominates but is inconsistently defined, and the proliferation of bespoke composites prevents cross-study comparison. The field needs shared, attacker-aware metrics computed from agent traces that capture progression, persistence and unauthorised action, and that can be integrated into continuous evaluation pipelines [1, 2].

**3. Empirical validation of defences in real deployments.** Many proposed controls and metrics are defined theoretically or evaluated only in lab settings or specific case studies, and defences against multi-agent and lifecycle threats are supported by comparatively weak evidence. Real-world, multi-agent deployment studies—with ground truth for emergent coordination behaviours, which the literature notes is often missing—are a priority [2, 16].

**4. Persistence, memory and post-exploitation.** Memory poisoning, delayed-effect manipulation and post-exploitation persistence are recognised as distinctive agentic risks but remain

under-explored, with few reproducible test-beds for agent tool misuse and limited study of fine-grained adversarial perturbations in retrieved content [4, 6].

**5. Coverage of emerging architectures and modalities.** Much of the empirical work centres on a few architectures (function-calling and MCP) and on cloud-based, text-only deployments. Emerging agent-to-agent protocols, edge and hybrid deployments, and multimodal and embodied agents are largely unevaluated, limiting the generalisability of current findings [14, 20, 37].

**6. Defences that target progression, not components.** The clearest design lesson is that hardening individual components is insufficient under composed attacks; defences should disrupt attack progression pathways. This reframes the research agenda from “make each component robust” to “break the chain”, and calls for progression-aware detection, containment and recovery [1, 14].

## 10 Discussion

Three cross-cutting observations emerge from the synthesis.

**Autonomy and tools, not text, drive agentic risk.** The threat distribution (Table 5) is dominated by categories that arise from autonomy, tool integration, retrieval and orchestration—supply-chain compromise, excessive agency, insecure output handling, indirect injection, tool-use abuse and memory poisoning—rather than by text-only failure modes. Agentic security is therefore best understood as a *systems* security problem in which the LLM is one component among many, and in which the dangerous step is almost always the transition from language to privileged action [1, 21].

**Threats are better characterised than defences.** The corpus contains rich, convergent threat taxonomies and a growing body of attack measurements, but defensive evidence is thinner, more theoretical and weaker at the multi-agent and lifecycle level. The concept co-occurrence in the synthesis (Figure 5) shows that questions, domains, threats and methods co-occur densely, while datasets and rigorous evaluation are comparatively sparse—an asymmetry that should guide where empirical effort is directed next.

**Standards are converging but academic mapping lags.** Practitioner frameworks—OWASP GenAI/LLM and agentic top-ten lists, NIST AI RMF, MITRE ATLAS, CSA MAESTRO—recur throughout the corpus, and the most mature contributions explicitly map their threat models and controls onto these frameworks and onto regulation such as the EU AI Act and ISO/IEC standards [2, 10, 11]. This convergence is encouraging, but the academic literature mapping these frameworks onto concrete agentic deployments—and validating that the mapped controls actually reduce measured risk—is still nascent.

## 11 Threats to Validity

Several factors qualify the conclusions of this review.

**Construct and selection validity.** The field is young and terminology is unsettled; “agentic AI”, “LLM agent” and “AI security” are used inconsistently across venues, so some relevant work may use vocabulary our queries did not capture, and some retrieved work may match on surface terms without substantive relevance. We mitigated this with a broad, multi-origin query strategy (Boolean, database-specific, generated and keyword queries) and with two complementary relevance-prioritisation configurations whose high agreement supports the stability of the candidate pool, but residual selection effects remain.

**Database and access coverage.** Records were retrieved programmatically from five queryable sources; while these collectively cover the relevant publishers and preprint servers, some venues (notably parts of Scopus, Web of Science, IEEE Xplore, the ACM Digital Library and publisher platforms) were represented only indirectly through indexing. Full-text retrieval succeeded for about 70% of the candidate pool, so a minority of relevant reports could not be assessed at full-text depth.

**Recency and publication bias.** The corpus is heavily weighted toward 2024–2026 and toward preprints and standards documents, reflecting the pace of the field. Preprints have not all been peer-reviewed, and several included studies are surveys or position papers rather than primary empirical work. The review is therefore a snapshot of a fast-moving area and may have a bias toward English-language and openly accessible sources.

**Synthesis validity.** Evidence was extracted into structured profiles and synthesised thematically; the prevalence figures are article-level counts and depend on how each report framed its own threats, methods and metrics. Numerical results in particular were drawn from heterogeneous evaluations with differing definitions of “success”, so the quantitative figures in Section 8 are indicative rather than directly comparable, and were audited against their source reports before use.

## 12 Conclusion

Agentic AI moves the security question from “what will the model say?” to “what will the system do?”. This review synthesised the recent literature on the security of agentic AI and LLM-based applications, following a transparent PRISMA-2020/PRISMA-S process that screened 6,221 unique records and retained 35 studies for in-depth synthesis. The evidence shows a coherent and rapidly maturing picture: the attack surface is defined by the architecture around the model—the prompt channel, memory, RAG and vector stores, tools and code execution, multi-agent orchestration, communication protocols and the AI supply chain—and the dominant threats are those amplified by autonomy and tool use, including direct and indirect prompt injection, jailbreaks, sensitive-information disclosure, poisoning across data/model/memory/retrieval, tool-use abuse, excessive agency, system-prompt leakage, insecure output handling and supply-chain compromise. Reported defences are most effective when layered into a defence-in-depth stack that spans prompt hygiene, model robustness, least-privilege tool and access control, orchestration- and lifecycle-level controls, governance, and continuous attacker-aware evaluation, and they are most clearly aligned to

OWASP, NIST, MITRE ATLAS and emerging regulation.

The strongest cross-cutting finding is also the most actionable: single-vector evaluation systematically underestimates real-world risk, because composed and chained attacks succeed far more often than isolated ones, and because the choice of integration architecture changes both the level and the locus of exposure. The field therefore needs benchmarks and metrics built around attack *progression*, empirical validation of defences in real multi-agent deployments, deeper study of persistence and memory, coverage of emerging protocols, edge and multimodal settings, and defences that break the attack chain rather than hardening components in isolation. Closing these gaps—measuring what actually breaks these systems, and defending the path rather than the parts—is the central task for securing the next generation of deployed agentic AI.

## References

- [1] Ali Dehghantanha; Sajad Homayoun. Sok: The attack surface of agentic ai - tools, and autonomy. *arXiv.org*, 2026. doi: 10.48550/arxiv.2603.22928. URL <https://doi.org/10.48550/arxiv.2603.22928>.
- [2] Shaina Raza; Ranjan Sapkota; Manoj Karkee; Christos Emmanouilidis. Trism for agentic ai: A review of trust, risk, and security management in llm-based agentic multi-agent systems. *AI Open*, 2026. doi: 10.1016/j.aiopen.2026.02.006. URL <https://doi.org/10.1016/j.aiopen.2026.02.006>.
- [3] Anshuman Chhabra; Shrestha Datta; Shahriar Kabir Nahin; Prasant Mohapatra. Agentic ai security: Threats, defenses, evaluation, and open challenges. *IEEE Access*, 2025. doi: 10.1109/access.2026.3675554. URL <https://doi.org/10.1109/access.2026.3675554>.
- [4] Hanrong Zhang; Jingyuan Huang; Kai Mei; Yifei Yao; Zhenting Wang; Chenlu Zhan; Hongwei Wang; Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents. *arXiv (Cornell University)*, 2024. doi: 10.48550/arxiv.2410.02644. URL <https://doi.org/10.48550/arxiv.2410.02644>.
- [5] Narek Maloyan; Dmitry Namiot. Prompt injection attacks on agentic coding assistants: A systematic analysis of vulnerabilities in skills, tools, and protocol ecosystems. *arXiv.org*, 2026. doi: 10.48550/arxiv.2601.17548. URL <https://doi.org/10.48550/arxiv.2601.17548>.
- [6] Xinyun Liu; Zelei Cheng; Haodong Zhao; Ronghua Xu. Security of large model-based agents: A survey on adversarial, poisoning, and backdoor attacks. 2026. doi: 10.36227/techrxiv.177006506.61959855/v1. URL <https://doi.org/10.36227/techrxiv.177006506.61959855/v1>.
- [7] Shaina Raza; Ranjan Sapkota; Manoj Karkee; Christos Emmanouilidis. Trism for agentic ai: A review of trust, risk, and security management in llm-based agentic multi-agent systems. *ArXiv.org*, 2025. doi: 10.48550/arxiv.2506.04133. URL <https://doi.org/10.48550/arxiv.2506.04133>.
- [8] Jinhu Qi; Muzhi Li; Jiahong Liu; Yuqin Shu; D S F Yu; Shicheng Ma; Wenqian Cui; Yiyang

- Zhao; Yuehua Chen; Ruoxi Jiang; Irwin King; Zenglin Xu. Towards trustworthy agentic ai: a comprehensive survey of safety, robustness, privacy, and system security. *Academia AI and Applications*, 2026. doi: 10.20935/acadai8260. URL <https://doi.org/10.20935/acadai8260>.
- [9] Zonghao Ying; Xiao Yang; Siyang Wu; Yumeng Song; Yang Qu; Hainan Li; Tianlin Li; Jiakai Wang; Aishan Liu; Xianglong Liu. Uncovering security threats and architecting defenses in autonomous agents: A case study of openclaw. *arXiv.org*, 2026. doi: 10.48550/arxiv.2603.12644. URL <https://doi.org/10.48550/arxiv.2603.12644>.
- [10] Amy Chang; Tiffany Saade; Sanket Mendapara; Adam Swanda; Ankit Garg. Cisco integrated ai security and safety framework report. *ArXiv.org*, 2025. doi: 10.48550/arxiv.2512.12921. URL <https://doi.org/10.48550/arxiv.2512.12921>.
- [11] Rah A. Enterprise generative ai: A systematic review of security risks and governance. *Research Square*, 2026. doi: 10.21203/rs.3.rs-9890887/v1. URL <https://doi.org/10.21203/rs.3.rs-9890887/v1>.
- [12] Jaqueline Damacena Duarte; Guilherme D. Cândido; José Ricardo A. De Britto Filho; João Souza Neto; Elena J. da Costa; J. P. J. da Costa; Laerte Peotta de Melo. A systematic review of prompt injection attacks on large language models: Trends, taxonomy, evaluation, defenses, and opportunities. *IEEE Access*, 2026. doi: 10.1109/access.2026.3656849. URL <https://doi.org/10.1109/access.2026.3656849>.
- [13] E. Mathew. Enhancing security in large language models: A comprehensive review of prompt injection attacks and defenses. *Journal of Artificial Intelligence*, 2025. doi: 10.32604/jai.2025.069841. URL <https://doi.org/10.32604/jai.2025.069841>.
- [14] Tarek Gasmi; Ramzi Guesmi; Ines Belhadj; Jihene Bennaceur. Bridging ai and software security: A comparative vulnerability assessment of llm agent deployment paradigms. 2025. doi: 10.36227/techrxiv.175339471.17113065/v1. URL <https://doi.org/10.36227/techrxiv.175339471.17113065/v1>.
- [15] Hanxiang Xu; Shenao Wang; Ningke Li; Kailong Wang; Yanjie Zhao; Kai Chen; Ting Yu; Yang Liu; Haoyu Wang. Large language models for cyber security: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3769676. URL <https://doi.org/10.1145/3769676>.
- [16] Tharakesvulu Vangalapat; Samreen Iftekhhar Shaikh. Trustworthy agentic ai: A survey and taxonomy of secure coordination and hallucination mitigation in multi-agent large language model systems. *International Journal of Innovative Science and Research Technology*, 2026. doi: 10.38124/ijisrt/26feb1090. URL <https://doi.org/10.38124/ijisrt/26feb1090>.
- [17] M. Nadeesh; S. Saranya; M. Nandhini; S. Saravana kumar; K. Kanniyarasu; M. Sakthivadivel. Prompt injection attacks and mitigation strategies in advanced large language model applications. *International Journal of Drug Delivery Technology*, 2026. doi: 10.25258/ijddt.16.52s.55. URL <https://doi.org/10.25258/ijddt.16.52s.55>.

- [18] Audrey Rah; Sven Hahues. Retrieval-augmented generation (rag), generative ai, and agentic ai governance: An integrated enterprise governance prioritization architecture. 2026. doi: 10.21203/rs.3.rs-10012178/v1. URL <https://doi.org/10.21203/rs.3.rs-10012178/v1>.
- [19] Igor; Hagag Ben; Anderson William L.; Belcak Peter; Bucknall Ben; Cai Xiaohong; Chopra Ayush; Cohen Doron; Del Rosario Ron F.; Draguns Andis; Gray Annie; Katz Keren; Mavroudis Vasilios; Mink Jaron; Motwani Sumeet Ramesh; Petit Jonathan; Rembeck Leif-Sebastian; Smith Chandler Christian Schroeder de Witt; Krawiecka, Klaudia; Krawczuk. Open challenges in multi-agent security: Towards secure systems of interacting ai agents. *arXiv (Cornell University)*, 2025. doi: 10.48550/arxiv.2505.02077. URL <https://doi.org/10.48550/arxiv.2505.02077>.
- [20] Dezhang Kong; Shi Lin; Zhenhua Xu; Zhebo Wang; Minghao Li; Yufeng Li; Yilun Zhang; Hujin Peng; Zeyang Sha; Yuyuan Li; Changting Lin; Xun Wang; Xuan Liu; Ningyu Zhang; Chao-Jun Chen; Muhammad Khurram Khan; Meng Han. A survey of llm-driven ai agent communication: Protocols, security risks, and defense countermeasures. *arXiv.org*, 2025. doi: 10.48550/arxiv.2506.19676. URL <https://doi.org/10.48550/arxiv.2506.19676>.
- [21] Vineeth Sai Narajala; Om Narayan. Securing agentic ai: A comprehensive threat model and mitigation framework for generative ai agents. *ArXiv.org*, 2025. doi: 10.48550/arxiv.2504.19956. URL <https://doi.org/10.48550/arxiv.2504.19956>.
- [22] Tarek Gasmi; Ramzi Guesmi; I. Belhadj; Jihene Bennaceur. Bridging ai and software security: A comparative vulnerability assessment of llm agent deployment paradigms. *Information Sciences*, 2025. doi: 10.48550/arxiv.2507.06323. URL <https://doi.org/10.48550/arxiv.2507.06323>.
- [23] Xinhao Deng; Yixiang Zhang; Jiaqi Wu; Jiaqi Bai; Sibao Yi; Zhuo Zou; Yue Xiao; Rennai Qiu; Jianan Ma; Jialuo Chen; Xiaohu Du; Xiaofan Yang; Shiwen Cui; Changhua Meng; Weiqiang Wang; Jiaking Song; Ke Xu; Qi Li. Taming openclaw: Security analysis and mitigation of autonomous llm agent threats. *arXiv.org*, 2026. doi: 10.48550/arxiv.2603.11619. URL <https://doi.org/10.48550/arxiv.2603.11619>.
- [24] ; Dmytro Sytnikov. Security challenges of llm integration in multi-tenant saas: Threats, vulnerabilities, and mitigations. *Advances in Knowledge-Based Systems Data Science and Cybersecurity*, 2026. doi: 10.54364/cybersecurityjournal.2026.3121. URL <https://doi.org/10.54364/cybersecurityjournal.2026.3121>.
- [25] Robert Kemp. Evaluating prompt injection defenses in large language models: A multi-model empirical study of security–usability trade-offs. *Journal of Information Technology, Cybersecurity, and Artificial Intelligence*, 2026. doi: 10.70715/jitcai.2026.v3.i3.071. URL <https://doi.org/10.70715/jitcai.2026.v3.i3.071>.
- [26] Madiha Fathima. Llm adversarial prompt attack detection and mitigation engine: A novel framework for securing generative ai systems. 2025. doi: 10.36227/techrxiv.175416873.36395198/v1. URL <https://doi.org/10.36227/techrxiv.175416873.36395198/v1>.

- [27] SUNIL KOTA. Systemic vulnerabilities in autonomous llm agents: A comprehensive threat analysis and authorization framework. *International Journal of Innovative Research in Engineering Multidisciplinary Physical Sciences*, 2025. doi: 10.37082/ijrmps.v13.i1.232960. URL <https://doi.org/10.37082/ijrmps.v13.i1.232960>.
- [28] IBRAHIM ADABARA; Bashir Olaniyi Sadiq; Aliyu Nuhu Shuaibu; Yale Ibrahim Danjuma; Maninti Venkateswarlu. Trustworthy agentic ai systems: a cross-layer review of architectures, threat models, and governance strategies for real-world deployment. *F1000Research*, 2025. doi: 10.12688/f1000research.169927.1. URL <https://doi.org/10.12688/f1000research.169927.1>.
- [29] Senthil Muthu. Unified security orchestration framework for protecting ai/ml systems against input-based and model supply chain attacks. *AI and Machine Learning Advances*, 2026. doi: 10.64220/amla.v2i2.003. URL <https://doi.org/10.64220/amla.v2i2.003>.
- [30] Ashwin Sharma; Anshul Goel. Adversarial threat simulation in large language models: Red teaming beyond prompt injection. *International Journal of Innovative Science and Research Technology*, 2025. doi: 10.38124/ijisrt/25nov556. URL <https://doi.org/10.38124/ijisrt/25nov556>.
- [31] Sadif; Farig Sadeque; Md Rizwan Parvez Asif Shahriar; Md. Mostafizur Rahman; Ahmed. A survey on agentic security: Applications, threats and defenses. *ArXiv.org*, 2025. doi: 10.48550/arxiv.2510.06445. URL <https://doi.org/10.48550/arxiv.2510.06445>.
- [32] Apurv Verma; Satyapriya Krishna; Sebastian Gehrmann; Madhavan Seshadri; Anu Pradhan; Tom Ault; Leslie Barrett; David Rabinowitz; John Doucette; NhatHai Phan. Operationalizing a threat model for red-teaming large language models (llms). *arXiv (Cornell University)*, 2024. doi: 10.48550/arxiv.2407.14937. URL <https://doi.org/10.48550/arxiv.2407.14937>.
- [33] Stephen Tete. Threat modelling and risk analysis for large language model (llm)-powered applications. *arXiv.org*, 2024. doi: 10.48550/arxiv.2406.11007. URL <https://doi.org/10.48550/arxiv.2406.11007>.
- [34] Leroy Jacob Valencia. Artificial intelligence as the new hacker: Developing agents for offensive security. *arXiv (Cornell University)*, 2024. doi: 10.48550/arxiv.2406.07561. URL <https://doi.org/10.48550/arxiv.2406.07561>.
- [35] Minrui Xu; Jiani Fan; Xinyu Huang; Conghao Zhou; Jiawen Kang; Dusit Niyato; Shiwen Mao; Han Zhu; Xuemin; Shen Shen; Kwok-Yan Lam. Forewarned is forearmed: A survey on large language model-based agents in autonomous cyberattacks. *ArXiv.org*, 2025. doi: 10.48550/arxiv.2505.12786. URL <https://doi.org/10.48550/arxiv.2505.12786>.
- [36] Mohamed Amine Ferrag; Fatima Alwahedi; Ammar Battah; Bilel Cherif; Abdechakour Mechri; Norbert Tihanyi; Tamás Bisztray; Mérouane Debbah. Generative ai in cybersecurity: A comprehensive review of llm applications and vulnerabilities. *Internet of Things and Cyber-Physical Systems*, 2025. doi: 10.1016/j.iotcps.2025.01.001. URL <https://doi.org/10.1016/j.iotcps.2025.01.001>.

- [37] Mohamed Amine Ferrag; Abderrahmane Lakas; Norbert Tihanyi; Merouane Abdelkader Debbah. Llm and ai agents for autonomous systems: A survey of applications, datasets, and security challenges. *IEEE Open Journal of Intelligent Transportation Systems*, 2026. doi: 10.1109/ojits.2026.3665677. URL <https://doi.org/10.1109/ojits.2026.3665677>.

# Security of agentic AI and LLM-based applications: a systematic literature review of threats, attack surfaces, benchmarks, and mitigation strategies

Literature search summary



**Figure 2:** Literature-search summary. (A) Search funnel from unique queries to unique retained records; (B) query-run outcomes (productive, zero-result, failed); (C) DOI completeness; (D–E) raw and unique records by database; (F) unique records by query origin; (G) year distribution of retained records; (H) metadata completeness; (I) run summary.

# Ranking validation dashboard: Security of agentic AI and LLM-based applications: a systematic literature review of threats, attack surfaces, benchmarks, and mitigation strategies

Top 1,000 records were evaluated using metadata completeness, rank-band scores, configured-term recovery, top-keyword rank decay, source contribution, query-origin contribution, and agreement between the two ranking strategies.



**Figure 3:** Ranking-validation dashboard. (A) candidate summary and shared top-N; (B) metadata completeness by field; (C) mean relevance score by rank band; (D) configured-term coverage by band; (E) keyword-frequency decay across bands; (F) database contribution; (G) query-origin contribution; (H) agreement between the two prioritisation configurations at successive cut-offs; (I) top validation terms.

## Full-text ranking dashboard: Security of agentic AI and LLM-based applications: a systematic literature review of threats, attack surfaces, benchmarks, and mitigation strategies

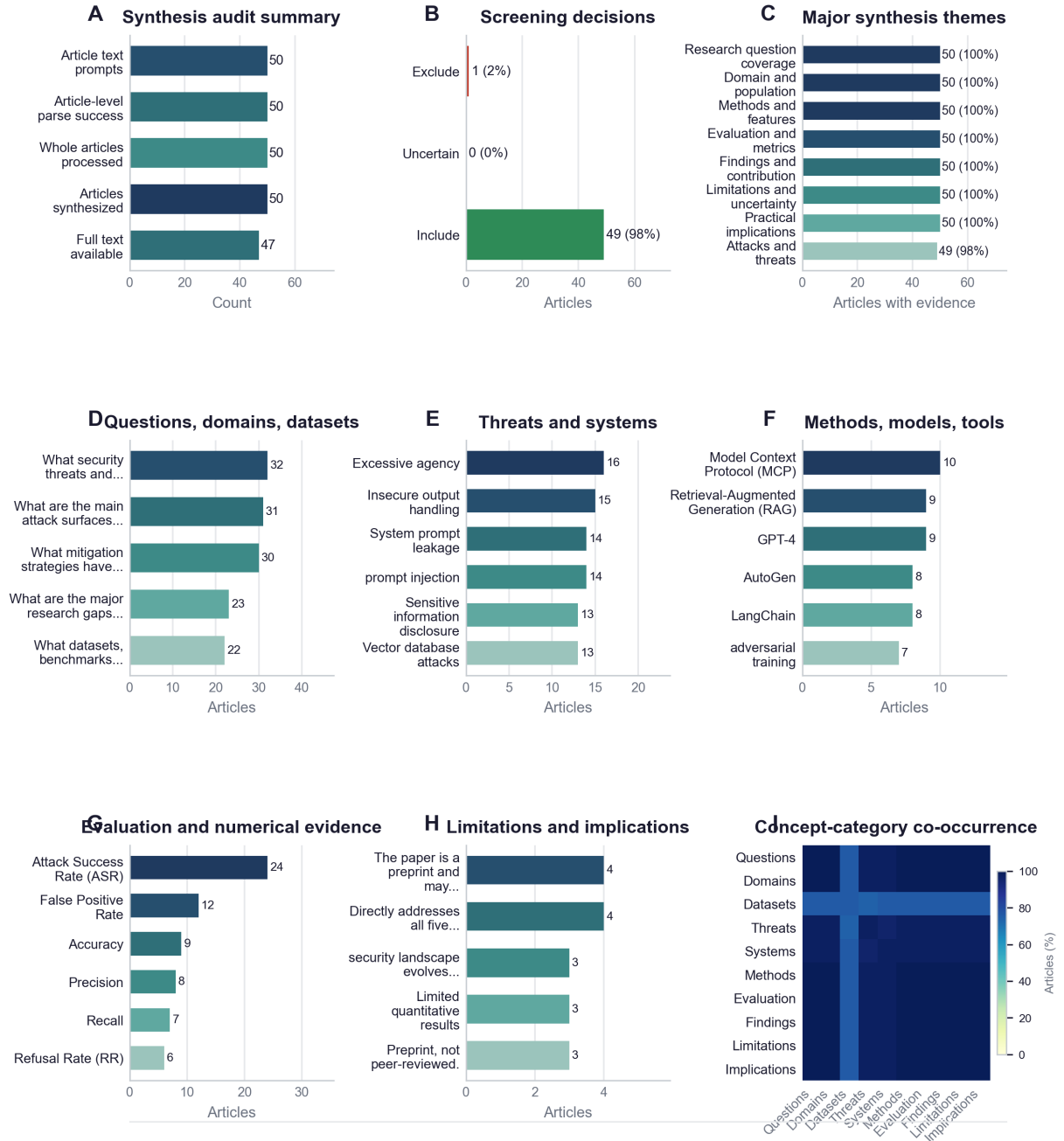
Top 200 full-text-ranked records were evaluated using full-text availability, rank-band scores, keyword rank decay, source contribution, query-origin contribution, and agreement with the preferred/hybrid ranking.



**Figure 4:** Full-text ranking dashboard for the top-200 assessment pool. (A) ranking summary; (B) full-text availability across ranking strategies; (C) mean score by band; (D) keyword-frequency decay; (E) database contribution; (F) query-origin contribution; (G) agreement of the selected hybrid ranking with alternative strategies; (H) top validation terms.

## Synthesis evidence dashboard: Security of agentic AI and LLM-based applications: a systematic literature review of threats, attack surfaces, benchmarks, and mitigation strategies

Article-level model-card outputs from deepseek-v4-flash were summarized for the top 50 full-text-ranked articles across screening decisions, major themes, extracted concepts, evaluation evidence, limitations, and category-level co-occurrence patterns.



Concept counts are article-level counts; one article contributes at most once per concept label.

**Figure 5:** Synthesis evidence dashboard over the in-depth assessment pool. (A) synthesis audit summary; (B) screening decisions; (C) major synthesis themes; (D) most-addressed questions, domains and datasets; (E) threats and systems; (F) methods, models and tools; (G) evaluation and numerical evidence; (H) limitations and implications; (I) concept-category co-occurrence.

### Synthesis concept summary: Security of agentic AI and LLM-based applications: a systematic literature review of threats, attack surfaces, benchmarks, and mitigation strategies

Top extracted concepts from 50 DeepSeek Flash article-level model cards. Each category lists frequent normalized concepts; bars show article-level counts.



Counts are article-level: one article contributes at most once to a concept label. Detailed concept nodes and co-occurrence edges are exported as CSV files.

**Figure 6:** Concept summary across the synthesised studies. Each category lists the most frequent normalised concepts; bars show article-level counts (one article contributes at most once per concept label).